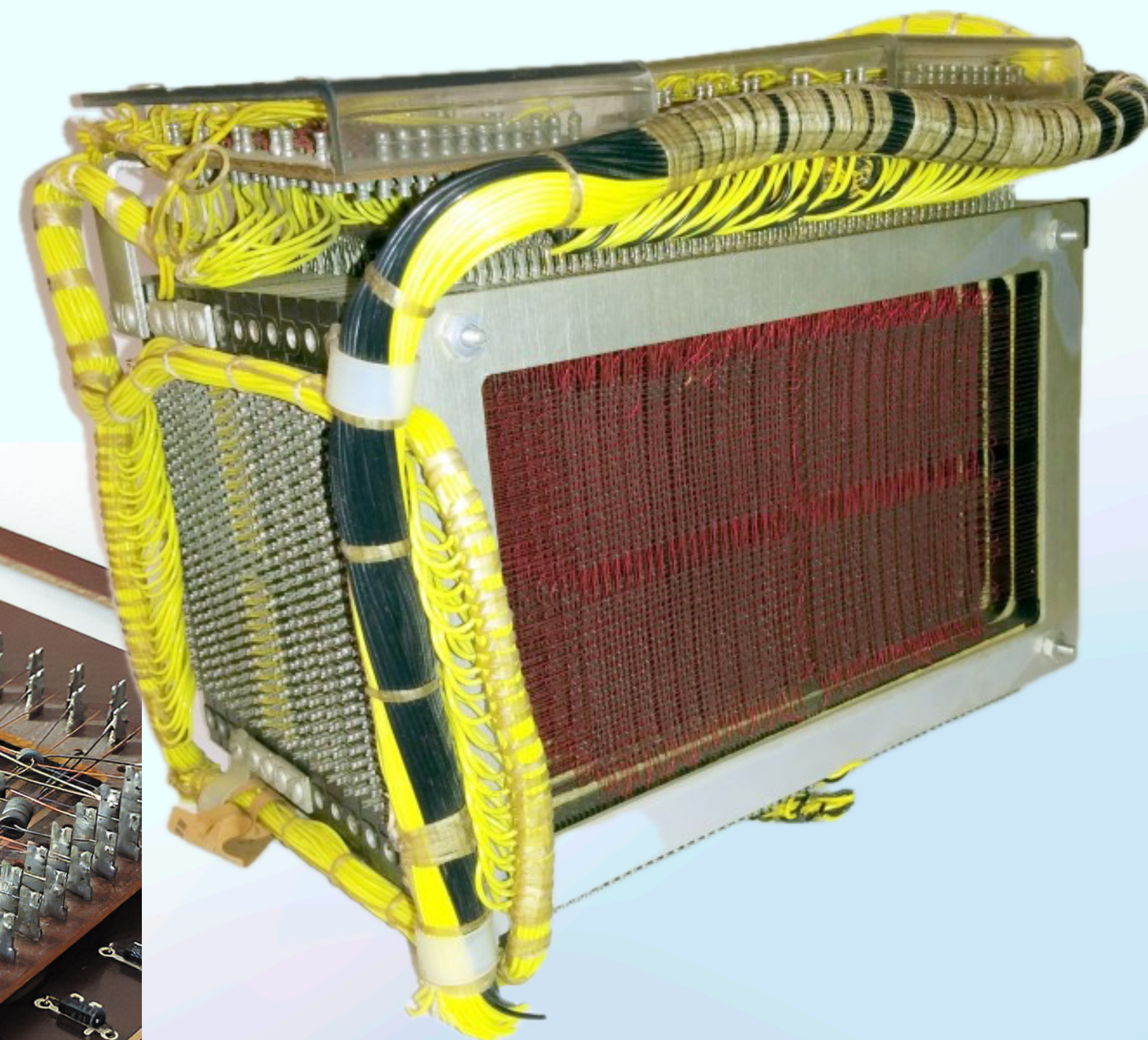
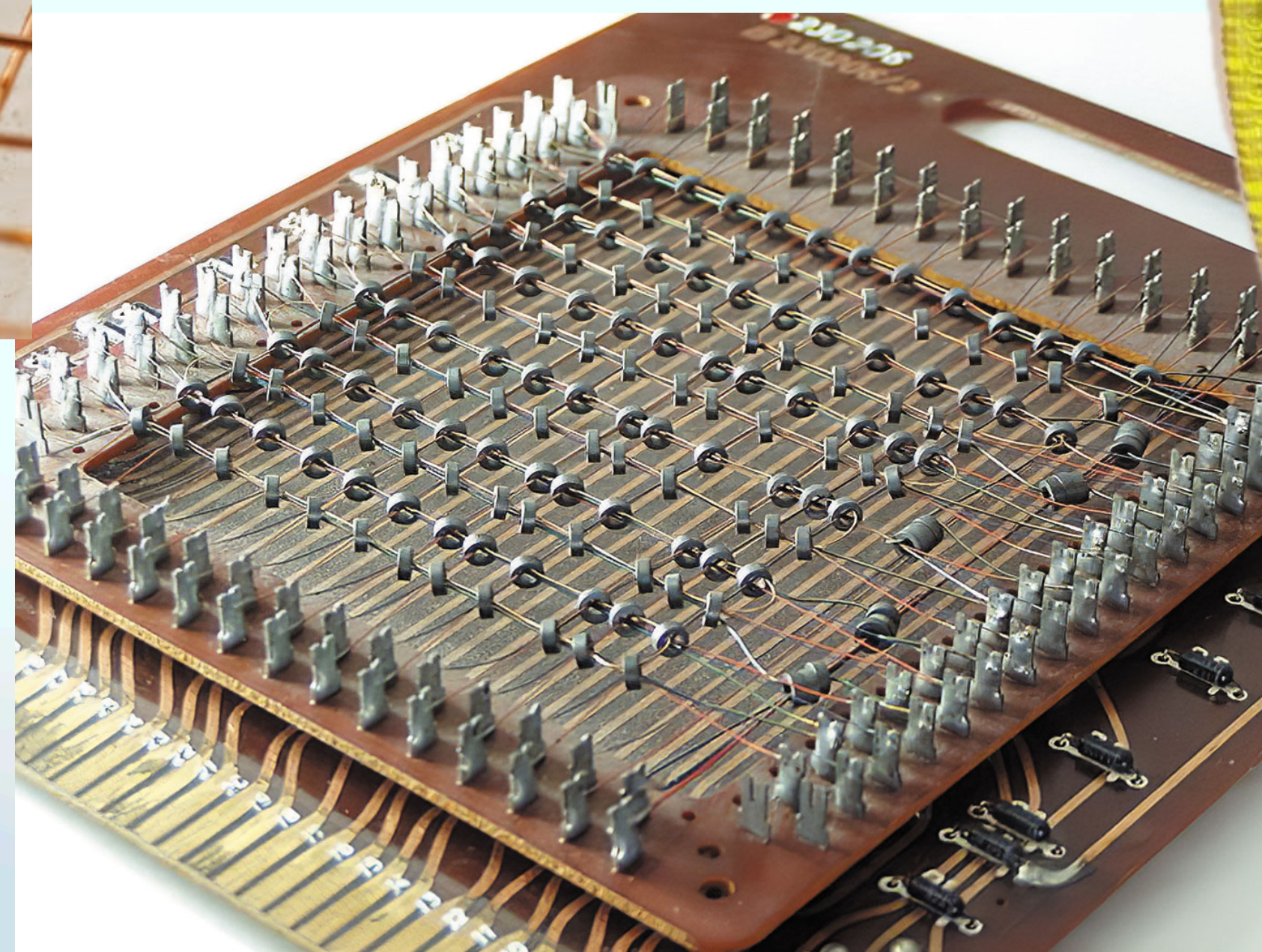
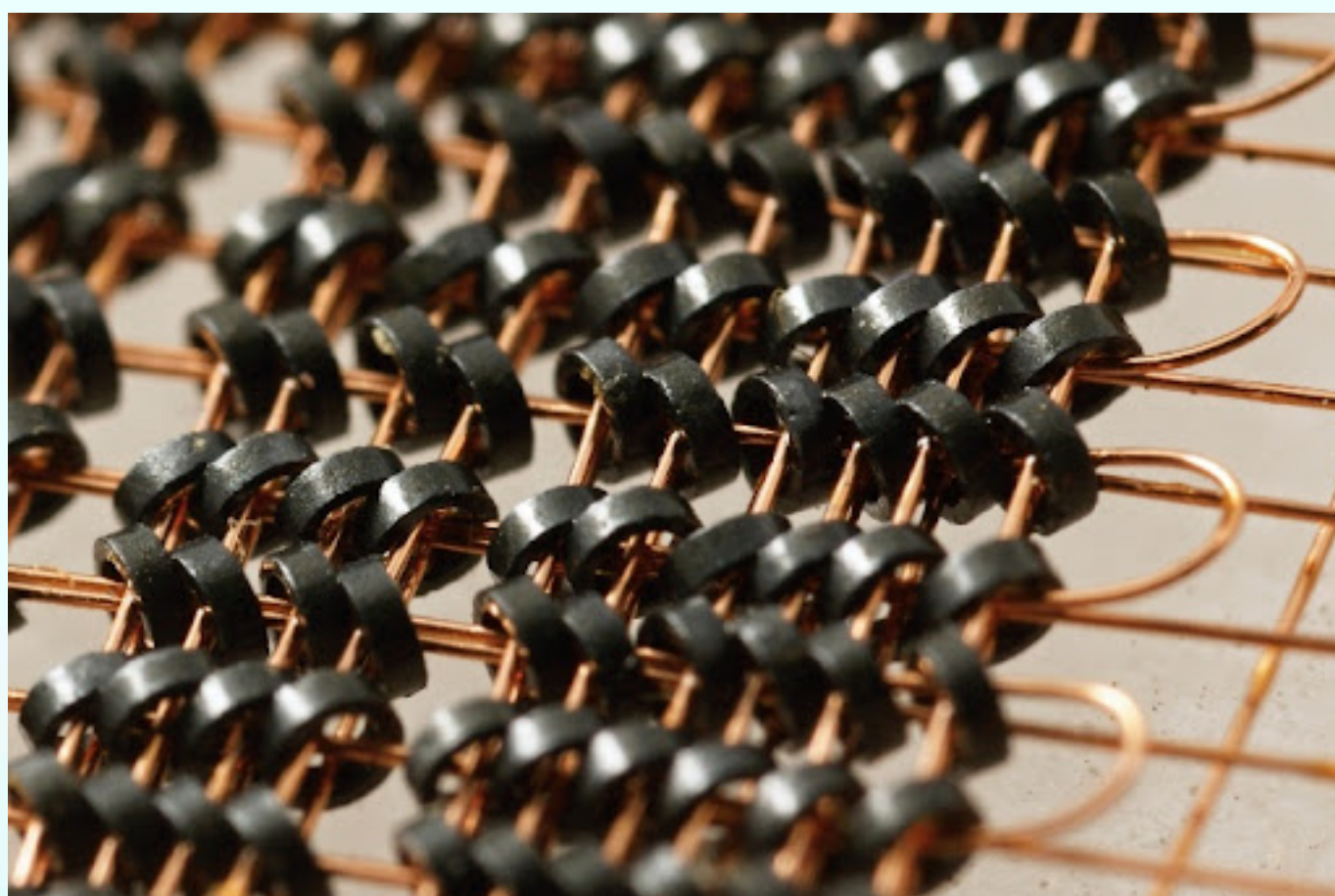


# Архитектура памяти

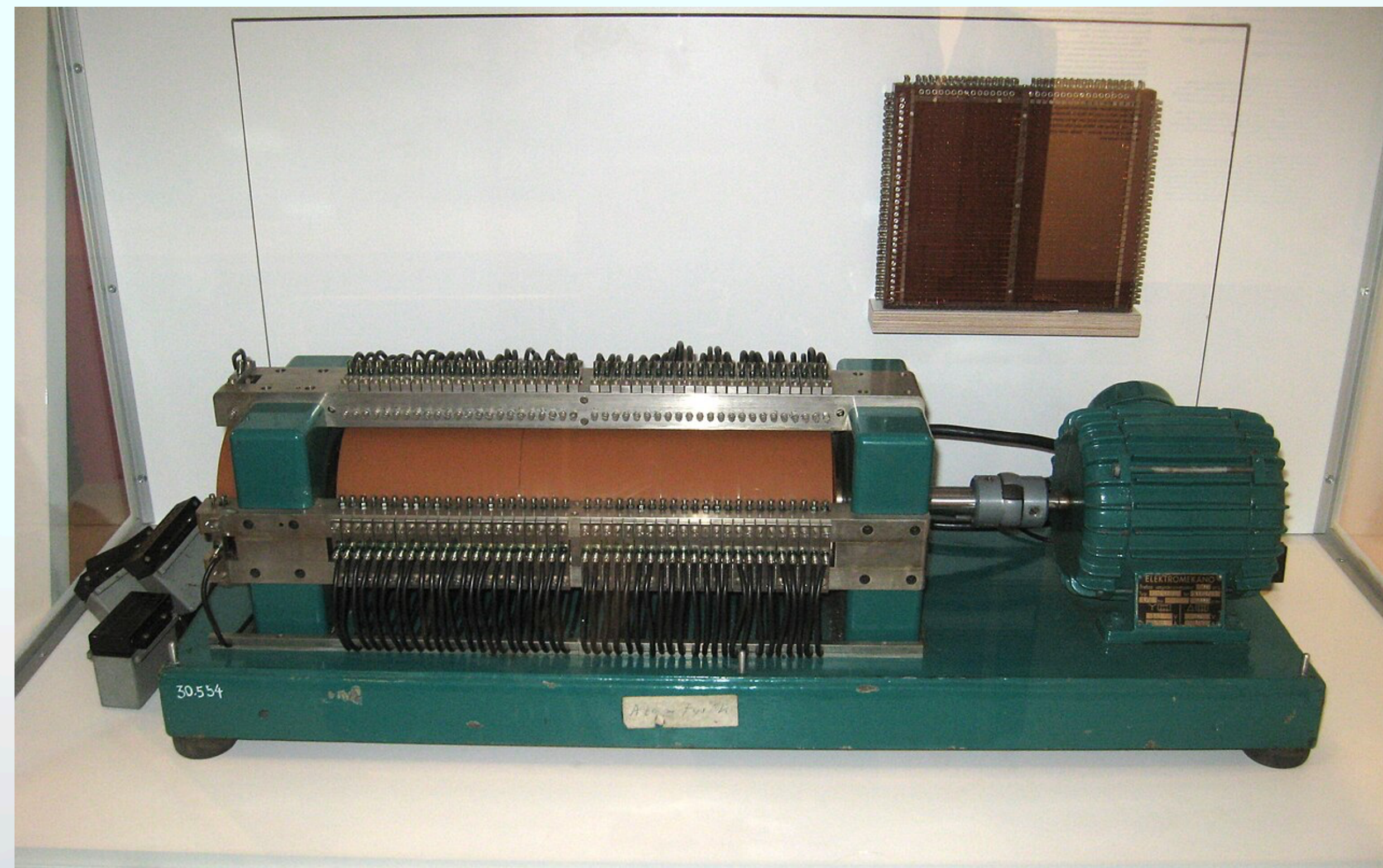
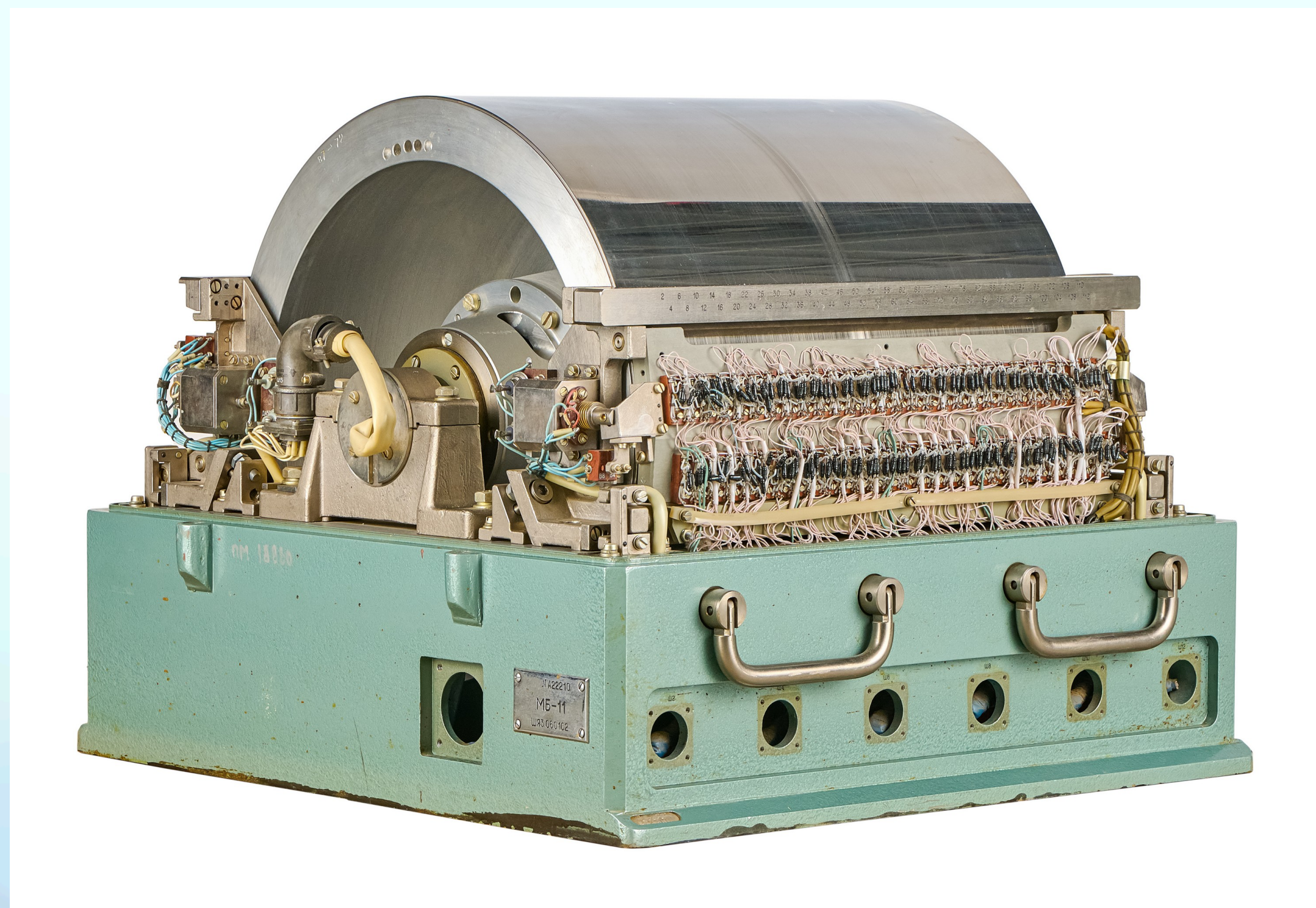


# Раньше





# Раньше





# Раньше

Память была очень дорогой и ее было мало. Если программа или ее данные не помещались в ОЗУ, программисту приходилось вручную писать код, который сохранял куски данных на магнитный барабан, а потом загружал их обратно. Ошибка в логике ломала всю систему, а перенос программы на компьютер с чуть большим объемом памяти требовал полной перегрузки и переписывания кода.



# История

- 1959–1962 гг. — Страничная организация
- 1965 г. — Сегментная организация
- Конец 1960-х — Сегментно-страничная организация



# Базовая концепция виртуальной памяти

Проблемы с физической памятью:

1. Проблема жестких адресов
2. Проблема безопасности
3. Физический потолок



# Базовая концепция виртуальной памяти

Виртуальная память — это слой абстракции между программами и реальными чипами ОЗУ.

- **Виртуальный адрес** — это адрес, который видит и использует программа в своем коде.
- **Физический адрес** — это реальное место (ячейка) внутри чипа оперативной памяти на материнской плате.



# Базовая концепция виртуальной памяти

1. Изоляция процессов
2. Динамическое перемещение
3. Выход за рамки ОЗУ (Файл подкачки/свап область)



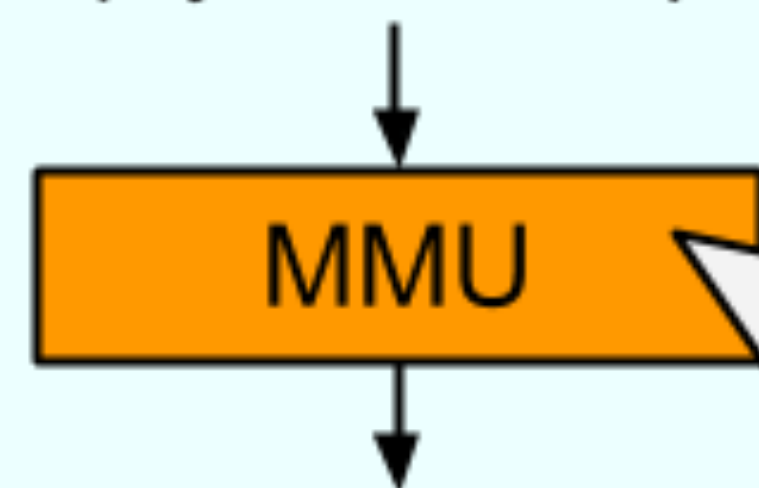
# Базовая концепция виртуальной памяти

MMU (Memory Management Unit)

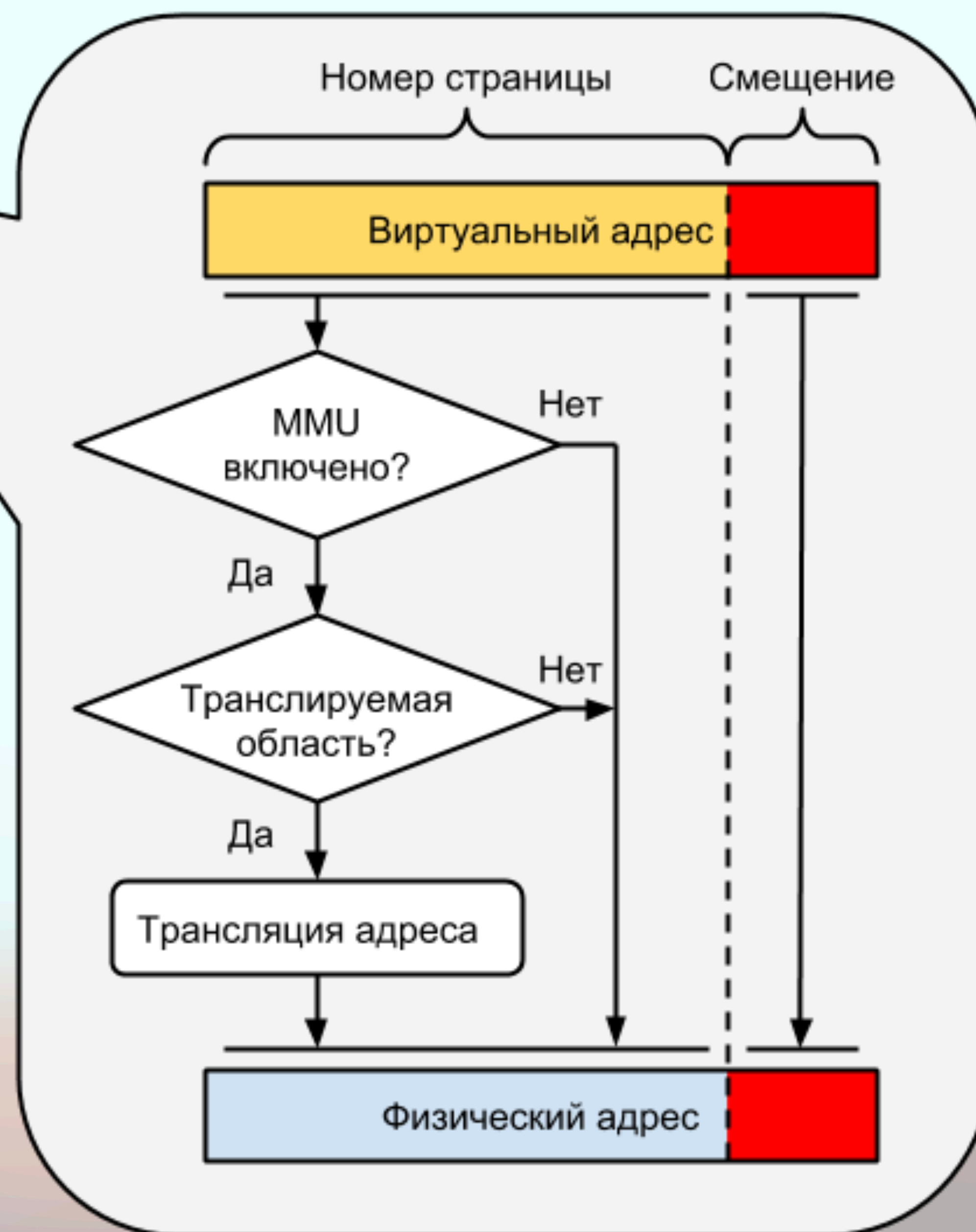
— это блок управления памятью. Это аппаратный компонент процессора, отвечающий за преобразование виртуальных адресов, используемых программами, в физические адреса оперативной памяти



Виртуальный адрес



Физический адрес





# Страничная организация

— это метод управления виртуальной памятью, при котором память программы разбивается на блоки фиксированного размера (страницы), а физическая память — на такие же блоки (кадры или страничные блоки), что позволяет операционной системе гибко распределять ресурсы и изолировать процессы

# Страничная организация

- **Виртуальная страница:** Кусочек фиксированного размера, на который нарезается виртуальная память процесса.
- **Физический фрейм:** Точно такой же по размеру кусочек, но уже внутри реальной физической оперативки. В физической памяти фреймы — это просто пронумерованные слоты.



# Страничная организация

## Пример

Программа состоит из трёх страниц:

- Страница 0 попадет во Фрейм 5
- Страница 1 попадет во Фрейм 2
- Страница 2 попадет во Фрейм 9

# Страничная организация

Таблица страниц

| Номер виртуальной страницы | Номер физического фрейма в ОЗУ | Статус         |
|----------------------------|--------------------------------|----------------|
| 0                          | Фрейм 5                        | В ОЗУ          |
| 1                          | Фрейм 2                        | В ОЗУ          |
| 2                          | Фрейм 9                        | В ОЗУ          |
| 3                          | Сброшена в файл подкачки       | На Диске (SSD) |



# Страничная организация

## Страничный сбой

Если страницы нет ОЗУ (она мб на диске), ОС копирует страницу в оперативную память и обновляет таблицу страниц. После чего программа продолжает выполняться с того же места, где и остановилась.

# Страничная организация

- + Нет внешней фрагментации
- Внутренняя фрагментация



# Сегментная организация

Программа никогда не состоит из однородной массы байт. С точки зрения программиста и компилятора, приложение — это набор логических блоков разного назначения и разного размера. Делит память процесса на сегменты, каждый из которых имеет свой смысл, свое имя и свой индивидуальный размер.

# Сегментная организация

1. Сегмент кода
2. Сегмент данных
3. Сегмент стека
4. Сегмент библиотек



# Сегментная организация

Поскольку сегменты имеют разную длину, виртуальный адрес в сегментной модели выглядит иначе. Он состоит из двух частей:

[ Номер сегмента | Смещение внутри сегмента ]

Вместо таблицы страниц операционная система создает Таблицу сегментов.

# Сегментная организация

| Номер сегмента | Описание             | Базовый адрес в ОЗУ | Предел (Размер сегмента) | Права доступа                 |
|----------------|----------------------|---------------------|--------------------------|-------------------------------|
| 0              | Код программы        | 10 000              | 1 500 байт               | Только чтение /<br>Выполнение |
| 1              | Глобальные<br>данные | 45 000              | 800 байт                 | Чтение / Запись               |
| 2              | Стек                 | 80 000              | 2 000 байт               | Чтение / Запись               |



# Сегментная организация

- + Жесткая безопасность и права доступа
- + Удобно делиться памятью
- Внешняя фрагментация памяти

# Сегментно-страничная организации памяти

Именно по этому принципу устроена работа оперативной памяти во всех современных компьютерах, смартфонах и серверах

Поскольку система стала двухступенчатой, виртуальный адрес теперь делится на три части:

[ Номер сегмента | Номер страницы внутри сегмента | Смещение на странице ]

# Сегментно-страничная организации памяти

Двойной перевод MMU

1. Проверка сегмента
2. Поиск физической страницы



# Сегментно-страничная организации памяти

- Больше нет внешней фрагментации
- Защита
- Файл подкачки стал эффективнее

# Многоуровневые таблицы и TLB

В теории сегментно-страничная модель выглядит идеально. Но когда инженеры попытались перенести её на современные 64-битные процессоры, они столкнулись с суровым математическим тупиком.

# Многоуровневые таблицы и TLB

Таблицы страниц стали слишком жирными

В старых 32-битных системах программа могла адресовать максимум 4 ГБ памяти. Если разбить 4 ГБ на страницы по 4 КБ, нам понадобится ровно 1 миллион строк в таблице страниц. Такая таблица весит около 4 МБ.

Если бы мы создали одну плоскую таблицу страниц под x64 (256Тб), она бы весила сотни Гигабайт! То есть вся оперативная память компьютера уходила бы только на хранение самой таблицы страниц для одной-единственной программы.



# Многоуровневые таблицы и TLB

## Многоуровневые таблицы страниц

Таблицу страниц превратили в многоуровневое дерево (в современных чипах Intel/AMD используется 4-уровневая или даже 5-уровневая структура).

# Многоуровневые таблицы и TLB

## Катастрофическая потеря скорости

Процессору нужно прочитать всего 1 байт данных из ОЗУ.

Но чтобы узнать, где этот байт лежит, блоку MMU приходится:

1. Сходить в ОЗУ за таблицей 1-го уровня.
2. Сходить в ОЗУ за таблицей 2-го уровня.
3. Сходить в ОЗУ за таблицей 3-го уровня.
4. Сходить в ОЗУ за таблицей 4-го уровня.
5. И только после этого сходить в ОЗУ за самим байтом!

# Многоуровневые таблицы и TLB

Катастрофическая потеря скорости

Чтобы процессор не бегал по древовидным таблицам при каждом запросе, инженеры встроили прямо внутрь процессора (в блок MMU) сверхбыструю специализированную память — TLB (Буфер перевода адресов).



# Итог

1. Программа видит красивую, непрерывную виртуальную память.
2. Код и данные игры разделены на сегменты
3. Эти сегменты разбиты на страницы по 4 КБ, чтобы железо могло раскидать их по ОЗУ как угодно, избегая фрагментации.
4. Адреса этих страниц хранятся в хитрых многоуровневых таблицах, чтобы экономить ОП.
5. Процессор летает на огромной скорости, потому что блок MMU находит адреса через супербыстрый кэш TLB.
6. А если игре не хватит вашей физической ОЗУ, операционная система незаметно отправит редкие страницы отдыхать в файл подкачки (свапе) на SSD.